Laporan Tugas Besar 2
IF2010 Pemrograman Berorientasi Objek 2025/2026
Banana Republic

Kode Kelompok : FRP
Nama Kelompok : Furap

1. 13524006 / Gabriella Botimada Lubis
2. 13524032 / Juan Oloando Simanungkalit
3. 13524064 / Stefani Angeline Oroh
4. 13524093 / Reinsen Silitonga
5. 13524098 / Reva Natania Sitohang



FRP - Furap

1. Deskripsi Umum Aplikasi

Banana Republic adalah aplikasi permainan strategi papan digital berbasis Java Desktop yang dibangun menggunakan JavaFX sebagai framework GUI. Permainan ini merupakan simulasi digital dari board game Catan yang diadaptasi dengan tema unik Kepulauan Banana, di mana 3 hingga 4 pemain (Nimon) bersaing untuk membangun peradaban, mulai dari menempatkan Pos Pantau dan Laboratorium, membangun jaringan Pipa Transportasi, mengumpulkan lima jenis sumber daya alam (Kayu, Batu Bata, Gandum, Bijih, Pisang), melakukan perdagangan domestik maupun maritim melalui pelabuhan, hingga memainkan Kartu Temuan Dr. Neroifa, untuk menjadi yang pertama meraih 10 Poin Prestasi. Seluruh mekanisme permainan, dari pengacakan dadu, aktivasi Nimon Ungu, sistem kartu spesial seperti Jalan Terpanjang & Pasukan Terbesar, hingga aturan perdagangan, diimplementasikan sepenuhnya dalam logika Java dengan antarmuka JavaFX yang interaktif.

Aplikasi ini dibangun di atas arsitektur berlapis yang memisahkan domain model, engine permainan, layer plugin, persistensi, dan UI secara tegas. Engine permainan menggunakan pola Strategy untuk mengelola fase giliran (Setup, Roll Dice, Trade/Build), pola Observer untuk menyiarkan event ke UI tanpa coupling langsung, dan sistem plugin berbasis URLClassLoader dan Java Reflection yang memungkinkan pemuatan kartu temuan baru, generator peta, maupun strategi bot dari file JAR eksternal tanpa restart aplikasi. Countdown timer giliran berjalan pada background thread terpisah dengan dispatch aman ke JavaFX Application Thread via Platform.runLater(), memastikan UI tetap responsif. State permainan dapat disimpan dan dimuat kembali dalam format JSON menggunakan library Gson.

2. Kakas GUI: JavaFX

1. JavaFX adalah framework GUI open-source berbasis Java untuk membangun aplikasi desktop lintas platform. Framework ini menjadi standar pengganti Swing dan mendukung rendering hardware-accelerated via Prism.
2. Lifecycle JavaFX: kelas utama meng-extend Application. Urutan: init() (inisialisasi non-UI di main thread) → start(Stage primaryStage) (entry point UI di JavaFX Application Thread, tempat Scene dan Stage dikonfigurasi) → stop() (cleanup saat aplikasi ditutup). Pemanggilan Application.launch() dari main() memulai seluruh lifecycle.
3. Komponen utama agar GUI muncul: (1) kelas extends Application sebagai entry point, (2) Stage sebagai jendela top-level, (3) Scene yang memuat root node scene graph, (4) pemanggilan stage.show(). Scene graph dibangun sebagai hierarki Node yang di-render oleh JavaFX runtime.

4. Komponen yang disediakan JavaFX: kontrol dasar (Button, Label, TextField, CheckBox, ComboBox), kontrol data (ListView, TableView, TreeView), layout pane (VBox, HBox, BorderPane, GridPane, StackPane, AnchorPane), Canvas untuk rendering kustom 2D, Dialog untuk popup, AnimationTimer dan Timeline untuk animasi, serta dukungan FXML untuk deklarasi UI terpisah dari logika Java dan CSS untuk styling.

3. Plugin & Class Loader

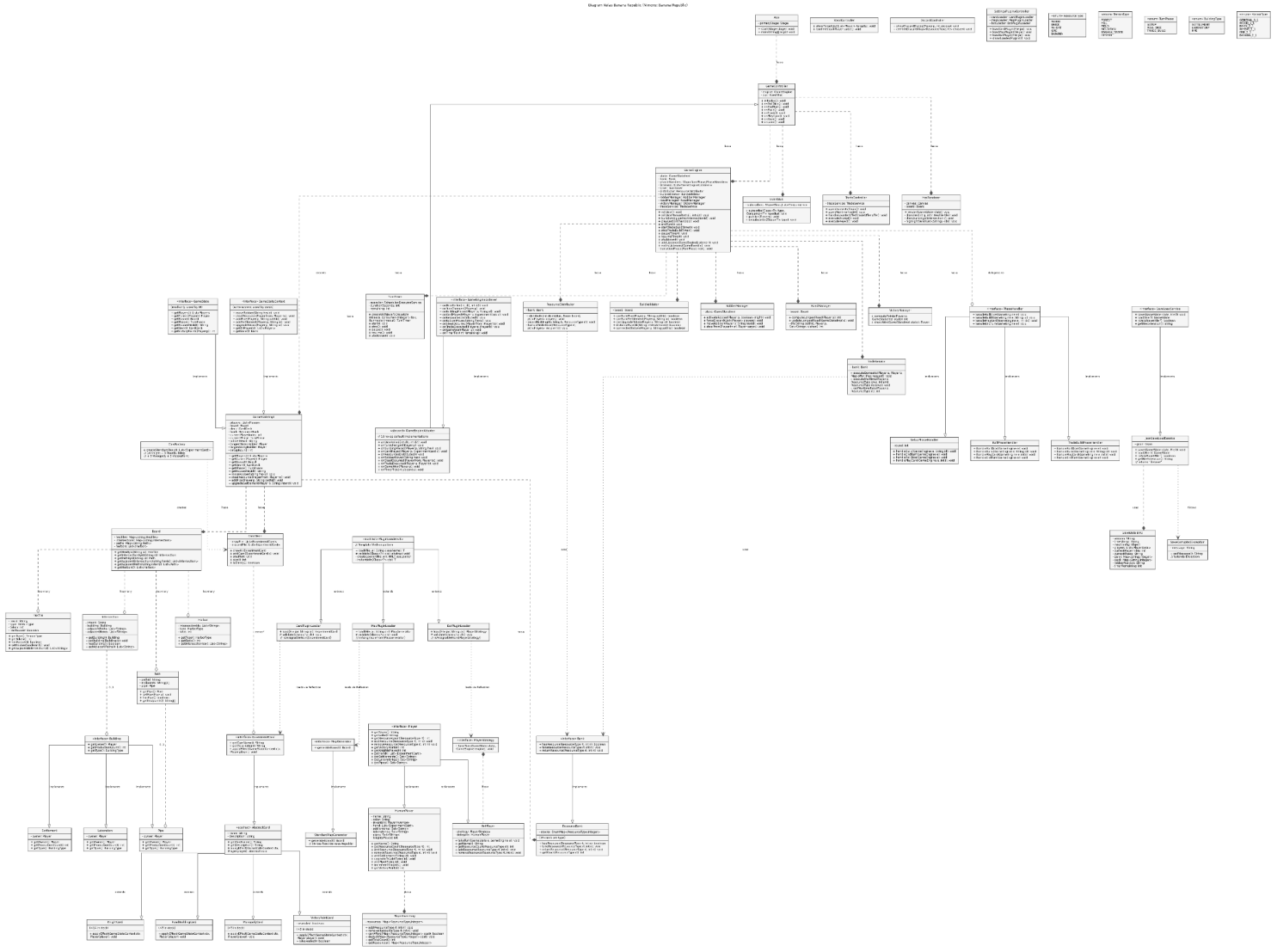
Sistem plugin kartu menggunakan URLClassLoader dari Java untuk memuat file .jar secara dinamis saat game berjalan tanpa perlu restart. Plugin harus mengimplementasikan interface ExperimentCard (src/main/java/model/card/ExperimentCard.java) yang mendefinisikan kontrak: getCardName(), getDescription(), dan applyEffect(GameStateContext ctx, Player player).

Mekanisme loading di CardPluginLoader (src/main/java/plugin/CardPluginLoader.java):

(1) URLClassLoader dibuat dengan URL file JAR.
(2) Class.forName(className, true, loader) memuat kelas dari JAR.
(3) ExperimentCard.class.isAssignableFrom(clazz) memvalidasi bahwa kelas mengimplementasikan interface yang benar, mencegah ClassCastException.
(4) clazz.getDeclaredConstructor().newInstance() menginstansiasi kartu via Reflection tanpa mengetahui nama konkret class-nya.
(5) Kartu ditambahkan ke CardDeck dan siap digunakan. Pola yang sama diterapkan di MapPluginLoader (untuk MapGenerator) dan BotPluginLoader (untuk PlayerStrategy).

Kelas abstrak PluginLoader<T> menjadi komponen generik pemuat plugin dengan pola Template Method. Subclass CardPluginLoader, BotPluginLoader, dan MapPluginLoader masing-masing menentukan interface target plugin lewat metode getPluginInterface(). Saat memuat plugin, URLClassLoader dibuat dengan URL berkas .jar yang ditunjuk, kemudian Class.forName() digunakan bersama loader tersebut untuk me-resolve kelas target. Java Reflection (getDeclaredConstructor().newInstance()) digunakan untuk membuat instance kelas yang dimuat tanpa mengetahui nama konkretnya secara statis. Validasi dilakukan melalui isAssignableFrom() untuk memastikan kelas yang dimuat mengimplementasikan interface yang benar, mencegah ClassCastException saat runtime. Kartu yang berhasil dimuat melalui CardPluginLoader langsung dimasukkan ke CardDeck aktif menggunakan injectCard() dan dikocok ulang.

4. Class Diagram



Proses pembuatan kelas diagram menggunakan **graphviz**. Detail desain kelas secara lengkap dapat dilihat pada lampiran kelas diagram.

Lapisan model berpusat pada interface Player yang diimplementasikan oleh HumanPlayer dan BotPlayer (extends HumanPlayer). HumanPlayer memiliki komposisi dengan PlayerInventory untuk mengelola sumber daya dan kartu. Hierarki kartu menggunakan interface ExperimentCard sebagai kontrak utama, AbstractCard sebagai kelas dasar bawaan, diikuti KnightCard, ProgressCard (kelas abstrak), serta subclass konkret MonopolyCard, RoadBuildingCard, dan VictoryPointCard. Bangunan dimodelkan melalui interface Building dengan implementasi Settlement dan Laboratory. Board mengandung koleksi HexTile, Intersection, BoardPath, dan Harbor.

Lapisan engine berpusat pada GameEngine yang memiliki komposisi dengan GameStateImpl (implementasi GameState dan GameStateContext), serta sub-manager seperti BuildValidator, ResourceDistributor, RobberManager, VictoryManager, RoadManager, dan TurnTimer. Fase giliran dimodelkan dengan interface PhaseHandler yang diimplementasikan oleh SetupPhaseHandler, RollPhaseHandler, dan TradeBuildPhaseHandler. GameEngine berkomunikasi dengan UI melalui interface GameEngineListener, dengan GameEngineAdapter sebagai abstract class helper untuk implementasi parsial. Lapisan plugin menggunakan kelas abstrak generik PluginLoader<T> dengan tiga subclass konkret. EventBus menggunakan pola Singleton untuk komunikasi event-driven antar komponen. Lapisan persistence menggunakan interface SaveLoadService yang diimplementasikan oleh JsonSaveLoadService.

5. Konsep OOP

5.1. Inheritance

Tuliskan daftar penggunaan konsep ini di aplikasi, misalnya:

1. BotPlayer (plugin/BotPlayer.java) extends HumanPlayer (model/player/HumanPlayer.java)
   BotPlayer mewarisi semua kapabilitas HumanPlayer (manajemen inventori, penempatan bangunan, pengelolaan special card) dan menambahkan field strategy bertipe PlayerStrategy. Inheritance di sini tepat karena BotPlayer IS-A HumanPlayer dari sisi domain game, ia berpartisipasi identik dalam semua mekanisme permainan, hanya pengambilan keputusannya yang berbeda.

2. KnightCard / VictoryPointCard (model/card/) extends AbstractCard implements ExperimentCard
   AbstractCard (model/card/AbstractCard.java) menyimpan cardName, description, dan category secara terpusat agar tidak ada duplikasi di tiap subclass (DRY). KnightCard dan VictoryPointCard cukup memanggil super() di konstruktor dan mengimplementasikan applyEffect() sesuai efek spesifiknya. AbstractCard sendiri implements ExperimentCard, sehingga semua subclass secara otomatis memenuhi kontrak interface.

3. MonopolyCard / RoadBuildingCard (model/card/) extends ProgressCard extends AbstractCard
   ProgressCard (model/card/ProgressCard.java) adalah kelas abstrak yang menjembatani AbstractCard dengan kartu-kartu bertipe PROGRESS. ProgressCard menetapkan category secara otomatis ke CardCategory.PROGRESS dan mengoverride isRevealedWhenPlayed() menjadi true. Ini membentuk rantai inheritance dua level: MonopolyCard -> ProgressCard -> AbstractCard -> ExperimentCard.

4. GameEngineAdapter (engine/GameEngineAdapter.java) extends (implements) GameEngineListener
   GameEngineAdapter adalah abstract class yang menyediakan implementasi kosong (no-op) untuk semua 16 metode interface GameEngineListener. Subclass UI Controller (seperti GameController) yang meng-extend Adapter hanya perlu meng-override metode yang benar-benar dibutuhkan, tanpa wajib mengimplementasikan seluruh interface. Ini adalah pola Adapter klasik untuk interface berukuran besar.

5. CardPluginLoader / BotPluginLoader / MapPluginLoader (plugin/) extends PluginLoader<T>
   PluginLoader<T> (plugin/PluginLoader.java) adalah kelas abstrak generik yang mengimplementasikan Template Method: metode final loadPlugin() dan loadAllFromJar() mendefinisikan alur loading (buat URLClassLoader, load kelas via Reflection, validasi, instantiate), sedangkan hook method getPluginInterface() yang abstract dikembalikan ke subclass untuk menentukan tipe plugin target.

5.2. Composition

1. GameEngine (engine/GameEngine.java) memiliki tepat satu GameStateImpl, satu Dice, satu TurnTimer, dan beberapa manager
   GameEngine adalah orchestrator utama permainan. Ia memiliki komposisi ketat dengan BuildValidator, ResourceDistributor, RobberManager, VictoryManager, RoadManager, masing-masing bertanggung jawab atas aspek spesifik logika permainan (SRP). GameEngine juga memiliki koleksi List<GameEngineListener> untuk notifikasi Observer.

2. GameStateImpl (engine/GameStateImpl.java) memiliki tepat satu Board, satu Bank, satu CardDeck, dan koleksi List<Player>
   GameStateImpl mengagregasi seluruh data state yang diperlukan selama permainan berlangsung. Board, Bank, CardDeck, dan list Player dibuat di luar GameStateImpl dan disuntikkan melalui konstruktor (Constructor Injection), mendukung prinsip Dependency Inversion.

3. HumanPlayer (model/player/HumanPlayer.java) memiliki tepat satu PlayerInventory dan koleksi Set<SpecialCard>
   PlayerInventory (model/player/PlayerInventory.java) mengenkapsulasi semua data inventori: EnumMap sumber daya, List<ExperimentCard>, dan hitungan knightCardsPlayed. Pemisahan ini mengikuti SRP, HumanPlayer menangani logika posisi bangunan dan status supply, sedangkan PlayerInventory fokus pada manajemen kartu dan sumber daya.

4. Board (model/board/Board.java) memiliki banyak HexTile, banyak Intersection, banyak BoardPath, dan banyak Harbor
   Board adalah agregat utama papan permainan. Ia menyimpan semua tiles, intersection (persimpangan hex), paths (jalur pipa), dan harbors sebagai koleksi yang di-pass melalui konstruktor. Board tidak pernah memodifikasi struktur tile secara langsung, produksi sumber daya dilakukan oleh ResourceDistributor menggunakan data dari Board.

5. CardDeck (model/card/CardDeck.java) memiliki Deque<ExperimentCard>

CardDeck menggunakan ArrayDeque<ExperimentCard> sebagai backing structure, memungkinkan operasi draw() dari atas deck dan injectCard() yang diikuti shuffle(). Penggunaan Deque mencerminkan karakteristik domain: kartu selalu diambil dari atas, injeksi plugin bisa dari bawah sebelum dikocok.

5.3. Interface

Interface	Keterangan
ExperimentCard(model/card/ExperimentCard.java)	Kontrak utama untuk semua kartu Temuan. Mendefinisikan getCardName(), getDescription(), applyEffect(GameStateContext, Player), getCategory(), dan default method isRevealedWhenPlayed(). Adalah titik extensi utama sistem plugin kartu.
Player(model/player/Player.java)	Kontrak pemain: getName(), getColor(), getResourceCount(), addResource(), removeResource(), canAfford(). Diimplementasikan oleh HumanPlayer. BotPlayer mewarisi melalui HumanPlayer. Interface ini juga digunakan sebagai parameter di GameState dan GameEngineListener sehingga UI tidak bergantung pada implementasi konkret.
Building(model/building/Building.java)	Kontrak bangunan: getOwnerName(), getVictoryPoints(), getResourceYield(), getBuildingTypeName(). Diimplementasikan Settlement (VP=1, yield=1) dan Laboratory (VP=2, yield=2). Memungkinkan VictoryManager dan ResourceDistributor bekerja secara polimorfis tanpa instanceof.
GameState(engine/GameState.java)	Read-only view dari state permainan yang diekspos ke komponen luar engine. Diimplementasikan GameStateImpl. Digunakan sebagai parameter di PhaseHandler, plugin PlayerStrategy, dan komponen UI, memastikan mereka hanya bisa membaca state tanpa mengubahnya secara bebas.
GameEngineListener(engine/GameEngineListener.java)	Pola Observer: mendefinisikan 16 callback event (onDiceRolled, onBuildingPlaced, onTurnChanged, onTimerTick, dll.). GameController meng-extend GameEngineAdapter untuk berlangganan event dari GameEngine dan memperbarui UI.

Bank(model/resource/Bank.java)	Kontrak bank sumber daya: getAvailable(), hasResource(), takeResource(), returnResource(). Diimplementasikan ResourceBank. Default method getCapacity() mengembalikan 19.
MapGenerator(model/board/MapGenerator.java)	Kontrak generator peta: generateBoard() dan getGeneratorName(). Diimplementasikan StandardMapGenerator. Adalah titik extensi sistem plugin peta (bonus).
SaveLoadService(persistence/SaveLoadService.java)	Kontrak layanan save/load: save(), load(), isValidSave(), getFileExtension(). Diimplementasikan JsonSaveLoadService. Interface ini memudahkan penggantian format save di masa mendatang (OCP).
PhaseHandler(engine/phase/PhaseHandler.java)	Kontrak handler fase giliran: handle(GameStateImpl, GameEngine) dan getHandledPhase(). Diimplementasikan SetupPhaseHandler, RollPhaseHandler, TradeBuildPhaseHandler. Memisahkan logika setiap fase dalam kelas terpisah.
PlayerStrategy(plugin/PlayerStrategy.java)	Kontrak strategi bot: takeTurn(GameState, Player) dan getStrategyName(). Dimuat secara runtime dari .jar eksternal melalui BotPluginLoader. BotPlayer memanggil strategy.takeTurn() saat giliran bot.

5.4. Method Overriding dan Method Overloading

5.4.1. Method Overriding

1. applyEffect() pada MonopolyCard vs RoadBuildingCard vs VictoryPointCard (model/card/)
   Ketiga kelas meng-override applyEffect(GameStateContext state, Player player) yang dideklarasikan di interface ExperimentCard. MonopolyCard mengimplementasikan logika penjaraahan sumber daya dari semua pemain lain; RoadBuildingCard membiarkan engine menangani interaksi pemilihan jalur pipa; VictoryPointCard tidak memiliki efek aktif karena VP dihitung oleh VictoryManager.

2. isRevealedWhenPlayed() pada VictoryPointCard dan KnightCard (model/card/)

Default method isRevealedWhenPlayed() di interface ExperimentCard mengembalikan true. VictoryPointCard meng-override-nya menjadi false (kartu PP rahasia tidak diperlihatkan). KnightCard meng-override-nya menjadi true secara eksplisit karena kartu ini harus ditampilkan ke semua pemain untuk penghitungan Pasukan Terbesar.

3. toString() padai berbagai kelas domain (Settlement, Laboratory (model/building/), HumanPlayer, BotPlayer (model/player/), AbstractCard (model/card/)
   Setiap kelas meng-override toString() dari Object untuk menampilkan informasi debugging yang bermakna. Sebagai contoh, BotPlayer.toString() menampilkan nama, nama strategi, dan warna; Settlement.toString() menampilkan nama pemilik.

4. getHandledPhase() padai SetupPhaseHandler vs RollPhaseHandler vs TradeBuildPhaseHandler (engine/phase/)
   Setiap implementasi PhaseHandler meng-override getHandledPhase() untuk mengembalikan TurnPhaseType yang sesuai, dan meng-override handle() untuk mengimplementasikan transisi fase yang spesifik.

5.4.2. Method Overloading

1. appendField() pada JsonSaveLoadService (persistence/JsonSaveLoadService.java)
   appendField() di-overload untuk tiga tipe nilai yaitu, appendField(sb, key, String value, depth), appendField(sb, key, int value, depth), dan appendField(sb, key, boolean value, depth). Ketiga versi memiliki signature berbeda namun melakukan operasi yang konseptual sama (menambahkan field JSON), sehingga pemanggil tidak perlu melakukan casting manual.

5.5. Polymorphism

Polymorphism	Keterangan
applyEffect() dipanggil secara polimorfis di GameEngine.playCard() (engine/GameEngine.java)	GameEngine menyimpan referensi bertipe ExperimentCard, namun saat playCard() dipanggil, metode card.applyEffect(state, player) dikirimkan secara dinamis ke implementasi konkret (KnightCard, MonopolyCard, dll.) tanpa

	engine perlu tahu tipe spesifiknya. Plugin kartu eksternal yang dimuat via URLClassLoader pun diperlakukan sama, polymorphism ini yang memungkinkan sistem plugin bekerja.
Building.getVictoryPoints() / getResourceYield() dipanggil di VictoryManager.computeVictoryPoints() dan ResourceDistributor	VictoryManager iterasi semua Intersection dan memanggil building.getVictoryPoints() tanpa instanceof. Settlement mengembalikan 1, Laboratory mengembalikan 2. Hal yang sama terjadi di ResourceDistributor, building.getResourceYield() menentukan berapa kartu sumber daya yang diterima pemain, polimorfis untuk Settlement dan Laboratory.
Player sebagai tipe parameter di GameEngineListener dan seluruh callback event	Semua callback GameEngineListener menerima Player (interface), bukan HumanPlayer/BotPlayer konkret. Di sisi UI, GameController meneruskan data Player ke GameRevisionView via interface GameViewCallback, yang mengakses getName(), getColor(), getTotalResourceCount() secara polimorfis, bekerja identik untuk pemain manusia maupun bot..
ExperimentCard sebagai elemen Deque<ExperimentCard> di CardDeck (model/card/CardDeck.java)	CardDeck menyimpan semua kartu sebagai ExperimentCard (interface). Saat draw() mengembalikan kartu, tipe konkretnya (KnightCard, MonopolyCard, dll.) tidak relevan bagi CardDeck, polimorfisme memungkinkan kartu plugin eksternal masuk ke dalam deck dan diperlakukan setara dengan kartu bawaan.
PlayerStrategy dipanggil di BotPlayer.executeTurn() (plugin/BotPlayer.java	BotPlayer menyimpan referensi strategy bertipe interface PlayerStrategy. Saat executeTurn() dipanggil, strategy.takeTurn(state, this) di-dispatch ke implementasi konkret yang dimuat dari .jar , bisa GreedyBot, AggressiveBot, atau strategi lain yang belum diketahui saat kompilasi.

5.6. Java API Collection

Java API Collection	Keterangan
EnumMap<ResourceType, Integer> di PlayerInventory dan ResourceBank	EnumMap dipilih karena key-nya adalah enum ResourceType. EnumMap dioptimasi secara internal menggunakan array, memberikan performa O(1) dengan overhead memori lebih rendah dibanding HashMap. PlayerInventory.getResourceSnapshot() mengembalikan Collections.unmodifiableMap(new EnumMap<>(resources)) sebagai view read-only untuk UI.
EnumSet<SpecialCard> di HumanPlayer (model/player/HumanPlayer.java)	heldSpecialCards diinisialisasi dengan EnumSet.noneOf(SpecialCard.class). EnumSet adalah implementasi Set paling efisien untuk enum, representasi bit-level membuatnya sangat cepat untuk operasi contains(), add(), dan remove().
Deque<ExperimentCard> (ArrayDeque) di CardDeck (model/card/CardDeck.java)	ArrayDeque digunakan sebagai double-ended queue yang mendukung pengambilan dari atas (pop()) dan penambahan di akhir (addLast()). Setelah injectCard(), Collections.shuffle() dipanggil pada List sementara untuk mengacak urutan kartu. Pilihan ArrayDeque memodelkan semantik deck kartu lebih tepat dibanding List.
Stream API di PlayerInventory.getCardsByCategory() dan HumanPlayer.getSpecialCardVictoryPoints()	getCardsByCategory() menggunakan stream().filter(c -> c.getCategory() == category).collect(Collectors.toList()) untuk memfilter kartu berdasarkan kategori. getSpecialCardVictoryPoints() menggunakan heldSpecialCards.stream().mapToInt(SpecialCard::getVictoryPoints).sum() untuk menjumlahkan poin dari special cards secara ekspresif.
ConcurrentHashMap di EventBus (plugin/EventBus.java)	EventBus menggunakan ConcurrentHashMap<Class<? extends GameEvent>, List<Consumer<? super GameEvent>>> untuk menyimpan subscriber per tipe event. ConcurrentHashMap dipilih karena EventBus diakses dari multiple thread (TurnTimer berjalan di background thread, UI di JavaFX Application Thread). computeIfAbsent() digunakan untuk membuat list subscriber jika

	belum ada, secara thread-safe.

5.7. SOLID

A. Single Responsibility Principle

- S1: ResourceDistributor (src/main/java/engine/ResourceDistributor.java) hanya bertanggung jawab mendistribusikan sumber daya ke pemain saat dadu dilempar, sedangkan orkestrasi giliran secara keseluruhan diatur oleh GameEngine (src/main/java/engine/GameEngine.java).
- S2: BuildValidator (src/main/java/engine/BuildValidator.java) hanya bertanggung jawab memvalidasi legalitas pembangunan (distance rule, konektivitas pipa, batas maksimal), sedangkan eksekusi pembangunan itu sendiri dilakukan oleh GameEngine.
- S3: PlayerInventory (src/main/java/model/player/PlayerInventory.java) hanya bertanggung jawab menyimpan dan mengelola resource pemain (add, remove, canAfford), sedangkan identitas dan bangunan pemain dikelola oleh HumanPlayer (src/main/java/model/player/HumanPlayer.java).

B. Open/Closed Principle

- O1: ExperimentCard (src/main/java/model/card/ExperimentCard.java) sebagai interface plugin kartu memungkinkan penambahan jenis kartu baru dari JAR eksternal tanpa mengubah satu baris pun kode CardDeck, GameEngine, atau CardFactory. Kartu baru cukup mengimplementasikan interface ini.
- O2: MapGenerator (src/main/java/model/board/MapGenerator.java) sebagai interface plugin peta memungkinkan penambahan algoritma peta baru tanpa mengubah kode Board atau logika game. StandardMapGenerator (src/main/java/model/board/StandardMapGenerator.java) adalah implementasi default-nya, dan plugin JAR dapat menyediakan implementasi alternatif.
- O3: PluginLoader (src/main/java/plugin/PluginLoader.java) sebagai abstract base class untuk semua loader — penambahan jenis plugin baru (kartu, peta, bot) hanya membutuhkan subclass baru (CardPluginLoader, MapPluginLoader, BotPluginLoader di src/main/java/plugin/) tanpa mengubah PluginLoader itu sendiri.

C. Liskov Substitution Principle

- L1: KnightCard, ProgressCard, RoadBuildingCard, MonopolyCard, dan VictoryPointCard (semua di src/main/java/model/card/) adalah subtype dari AbstractCard yang mengimplementasikan ExperimentCard. CardDeck menyimpan koleksi ExperimentCard

dan memanggil applyEffect() secara polimorfis — setiap subtype memenuhi kontrak interface tanpa mengubah perilaku yang diharapkan CardDeck.

- L2: HumanPlayer (src/main/java/model/player/HumanPlayer.java) dan BotPlayer (src/main/java/plugin/BotPlayer.java) keduanya mengimplementasikan Player (src/main/java/model/player/Player.java). GameEngine hanya berinteraksi via tipe Player — dapat menerima keduanya tanpa perubahan perilaku dari sisi engine.
- L3: ResourceBank (src/main/java/model/resource/ResourceBank.java) mengimplementasikan Bank (src/main/java/model/resource/Bank.java). GameEngine dan plugin ExperimentCard hanya bergantung pada tipe Bank, sehingga implementasi bank alternatif dapat disubstitusi tanpa mengubah kode yang bergantung padanya.

D. Interface Segregation Principle

- I1: GameState (src/main/java/engine/GameState.java) dan GameStateContext (src/main/java/model/card/GameStateContext.java) dipisah menjadi dua interface. UI hanya mendapat GameState (read-only, 13 method), sedangkan plugin ExperimentCard mendapat GameStateContext untuk memodifikasi state — tidak ada pihak yang dipaksa mengimplementasikan method yang tidak relevan bagi perannya.
- I2: GameEngineListener (src/main/java/engine/GameEngineListener.java) mendefinisikan 16 callback event. GameEngineAdapter (src/main/java/engine/GameEngineAdapter.java) menyediakan implementasi kosong untuk semua method sehingga subscriber cukup meng-override callback yang dibutuhkan saja, tidak dipaksa mengimplementasikan semua 16 method sekaligus.
- I3: SaveLoadService (src/main/java/persistence/SaveLoadService.java) dan ExperimentCard (src/main/java/model/card/ExperimentCard.java) masing-masing mendefinisikan interface dengan method minimal dan terfokus, di mana SaveLoadService hanya kontrak save/load dan ExperimentCard mendefinisikan 4 method kartu (getCardName(), getDescription(), applyEffect(), getCategory()) dan 1 default method (isRevealedWhenPlayed()), sehingga implementor tidak terbebani method yang tidak relevan dengan tanggung jawabnya.

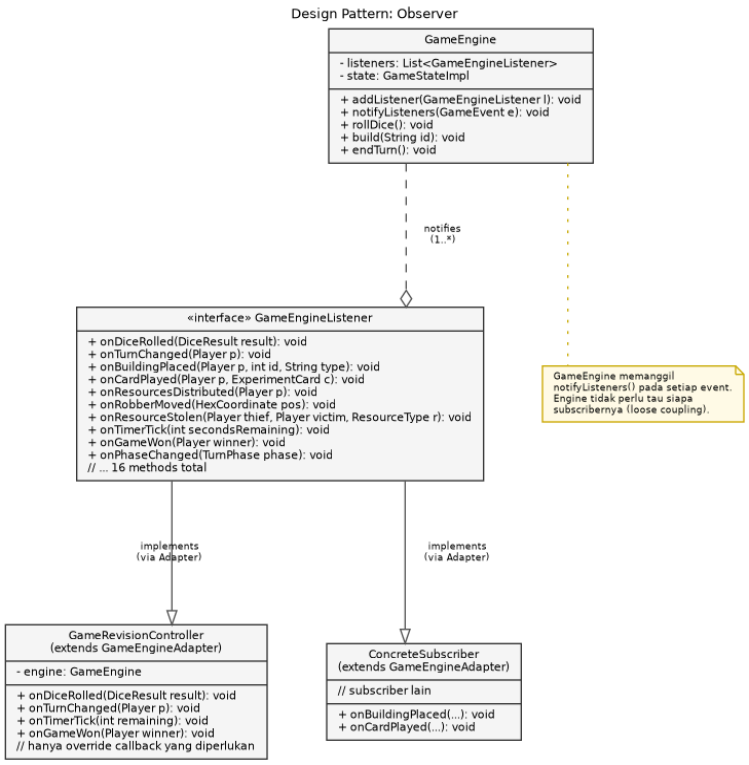
E. Dependency Inversion Principle

- D1: GameEngine (src/main/java/engine/GameEngine.java) bergantung pada abstraksi Bank (interface) bukan ResourceBank (konkret), dan GameState (interface) bukan GameStateImpl (konkret). High-level module tidak bergantung langsung pada detail implementasi.
- D2: CardDeck (src/main/java/model/card/CardDeck.java) menyimpan koleksi bertipe ExperimentCard (interface), bukan AbstractCard (abstract class konkret). Deck tidak perlu mengetahui tipe konkret kartu yang disimpannya, termasuk kartu yang di-load dari plugin JAR eksternal.

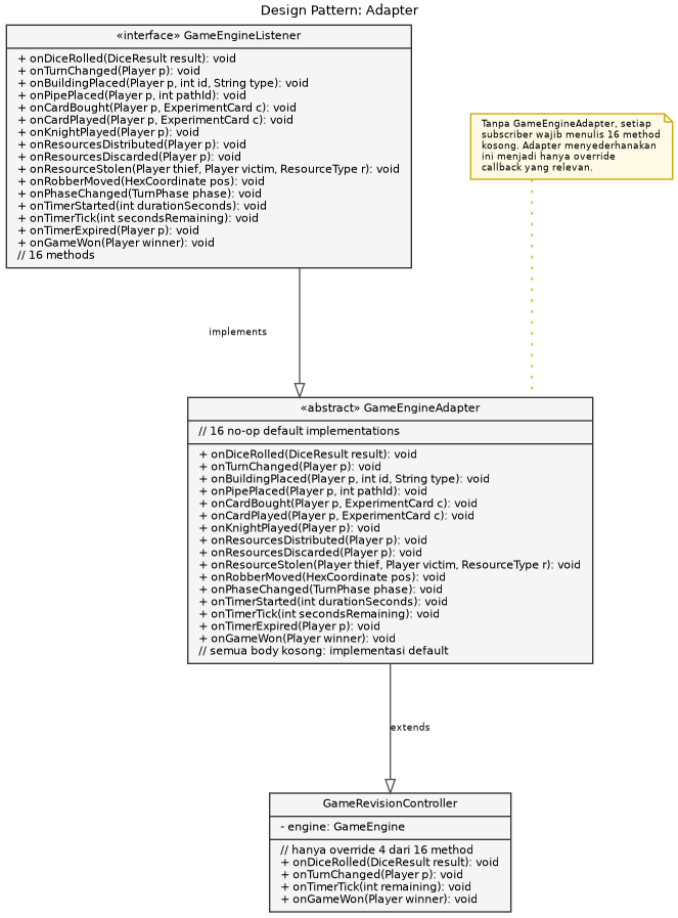
- D3: CardPluginLoader (src/main/java/plugin/CardPluginLoader.java) memvalidasi class yang di-load dengan ExperimentCard.class.isAssignableFrom(clazz) — loader bergantung pada interface, bukan class konkret. JAR eksternal apapun yang mengimplementasikan ExperimentCard dapat di-load tanpa loader perlu mengetahui nama class konkret di dalam JAR.

5.8. Design Pattern

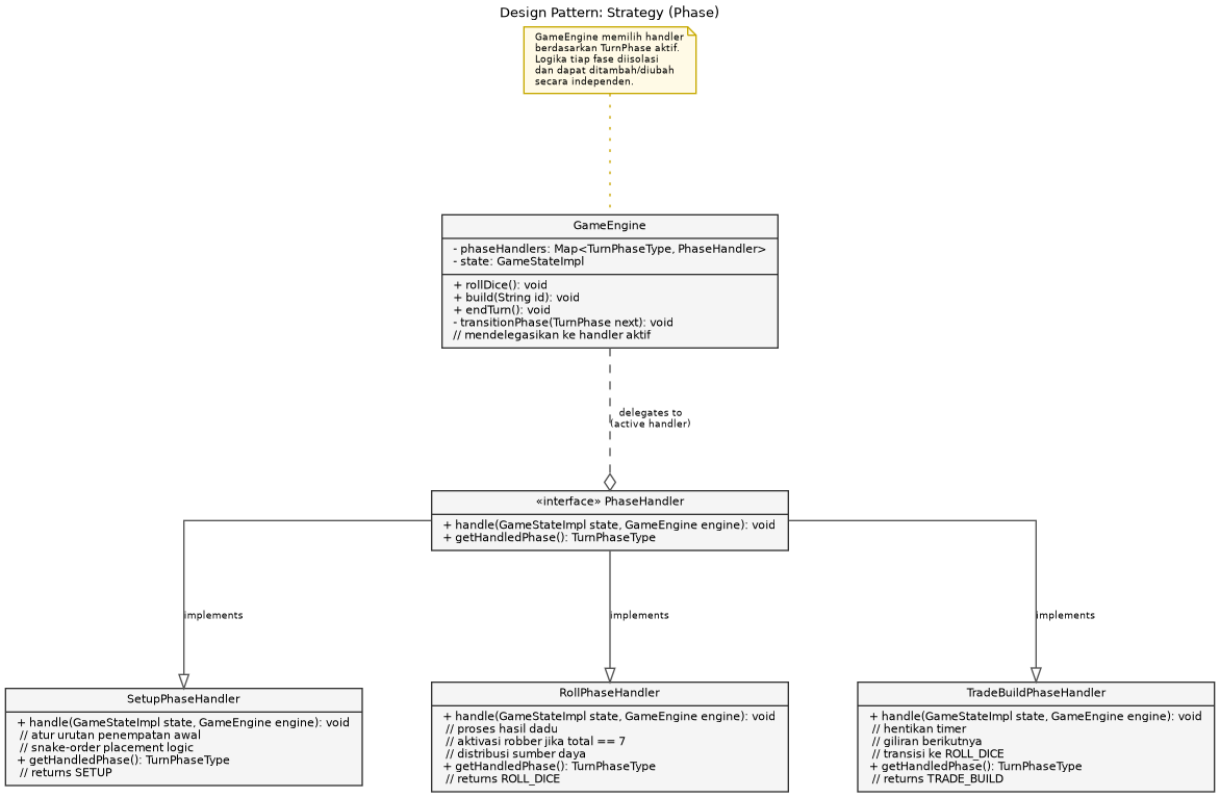
- **Observer Pattern**: GameEngine (src/main/java/engine/GameEngine.java) sebagai Subject menyimpan List<GameEngineListener> dan memanggil notifyListeners() pada setiap event game. Subscriber me-extend GameEngineAdapter (src/main/java/engine/GameEngineAdapter.java) dan hanya override callback yang dibutuhkan. Manfaat: engine tidak perlu mengetahui siapa subscriber-nya, coupling engine-UI diminimalkan.



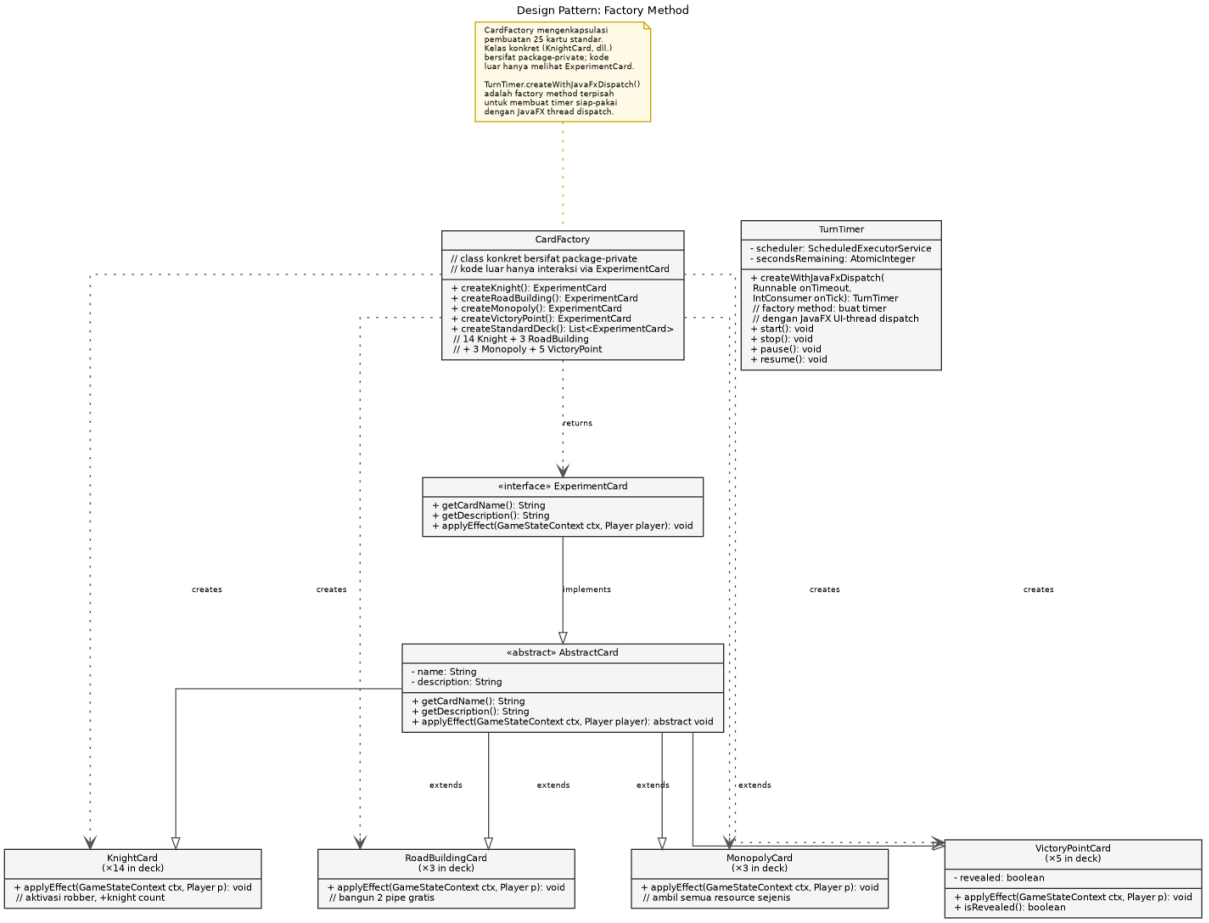
- **Adapter Pattern**: GameEngineAdapter (src/main/java/engine/GameEngineAdapter.java) adalah abstract class yang mengimplementasikan semua 16 method GameEngineListener dengan body kosong, bertindak sebagai adapter antara interface listener dan subscriber konkret. Subscriber cukup extend adapter dan override callback yang relevan tanpa menulis 16 implementasi kosong.



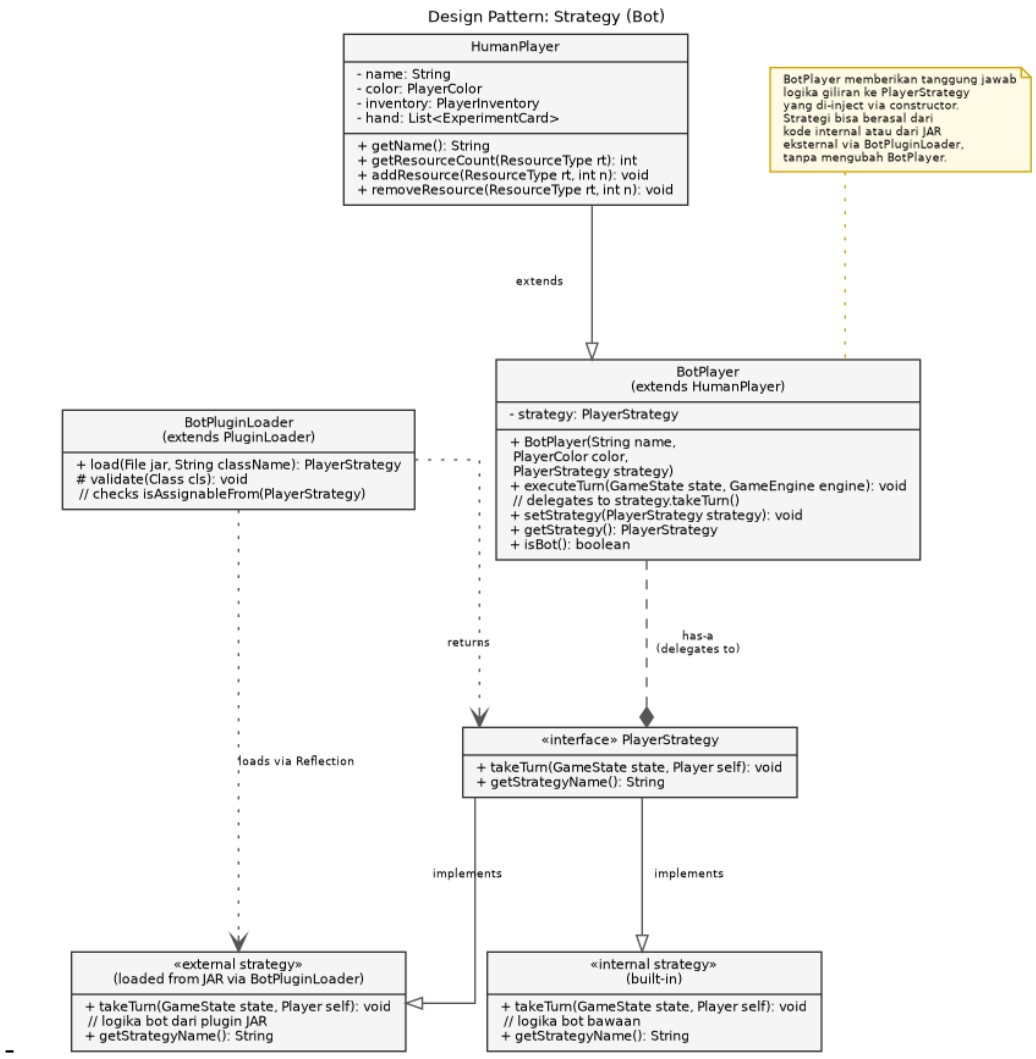
- **Strategy Pattern**: PhaseHandler (src/main/java/engine/phase/PhaseHandler.java) sebagai interface strategi dengan tiga implementasi: SetupPhaseHandler, RollPhaseHandler, TradeBuildPhaseHandler (src/main/java/engine/phase/). GameEngine mendelegasikan aksi ke handler yang aktif sesuai fase, sehingga logika tiap fase terisolasi dan dapat diubah/ditambah secara independen.



- **Factory Method Pattern**: CardFactory.createStandardDeck() (src/main/java/model/card/CardFactory.java) mengenkapsulasi pembuatan 25 kartu standar. Class konkret kartu (KnightCard, MonopolyCard, RoadBuildingCard, VictoryPointCard) dideklarasikan public final, tetapi kode luar hanya berinteraksi via ExperimentCard. TurnTimer.createWithJavaFxDispatch() (src/main/java/engine/TurnTimer.java) menggunakan factory method yang sama untuk membuat timer dengan JavaFX dispatch.



- **Strategy Pattern (Bot)**: PlayerStrategy (src/main/java/plugin/PlayerStrategy.java) sebagai interface dengan takeTurn(GameState, Player). BotPlayer (src/main/java/plugin/BotPlayer.java) mendelegasikan giliran ke PlayerStrategy yang di-inject via constructor. Strategi dapat disediakan internal atau di-load dari JAR via BotPluginLoader tanpa mengubah BotPlayer.


Design Pattern: Strategy (Bot)
HumanPlayer
- name: String
- color: PlayerColor
- inventory: PlayerInventory
- hand: List<ExperimentCard>
+ getName(): String
+ getResourceCount(ResourceType rt): int
+ addResource(ResourceType rt, int n): void
+ removeResource(ResourceType rt, int n): void
BotPlayer memberikan tanggung jawab logika giliran ke PlayerStrategy yang di-inject via constructor. Strategi bisa berasal dari kode internal atau dari JAR eksternal via BotPluginLoader, tanpa mengubah BotPlayer.
extends
BotPlayer
(extends HumanPlayer)
- strategy: PlayerStrategy
+ BotPlayer(String name, PlayerColor color, PlayerStrategy strategy)
+ executeTurn(GameState state, GameEngine engine): void
// delegates to strategy.takeTurn()
+ setStrategy(PlayerStrategy strategy): void
+ getStrategy(): PlayerStrategy
+ isBot(): boolean
BotPluginLoader
(extends PluginLoader)
+ load(File jar, String className): PlayerStrategy
validate(Class cls): void
// checks isAssignableFrom(PlayerStrategy)
returns
has-a
(delegates to)
loads via Reflection
«interface» PlayerStrategy
+ takeTurn(GameState state, Player self): void
+ getStrategyName(): String
implements
implements
«external strategy»
(loaded from JAR via BotPluginLoader)
+ takeTurn(GameState state, Player self): void
// logika bot dari plugin JAR
+ getStrategyName(): String
«internal strategy»
(built-in)
+ takeTurn(GameState state, Player self): void
// logika bot bawaan
+ getStrategyName(): String

-

5.9. Reflection

- Method loadAndInject() dan loadAllAndInject() pada CardPluginLoader (src/main/java/plugin/CardPluginLoader.java) menggunakan Reflection: URLClassLoader dengan URL JAR, Class.forName() untuk memuat kelas, isAssignableFrom(ExperimentCard.class) untuk validasi, getDeclaredConstructor().newInstance() untuk instansiasi. Memungkinkan kartu baru di-load tanpa mengetahui nama class konkretnya.
- Method loadAndGenerate() dan loadFirstGenerator() pada MapPluginLoader (src/main/java/plugin/MapPluginLoader.java) menggunakan mekanisme Reflection yang sama untuk memuat implementasi MapGenerator dari JAR eksternal, memvalidasi dengan MapGenerator.class.isAssignableFrom(clazz) sebelum instansiasi.
- Method loadAllStrategies() pada BotPluginLoader (src/main/java/plugin/BotPluginLoader.java) menggunakan Reflection untuk memuat PlayerStrategy dari JAR eksternal, memungkinkan penambahan strategi bot baru tanpa modifikasi aplikasi utama.

5.10. Multithreading

- TurnTimer (src/main/java/engine/TurnTimer.java) menjalankan countdown 90 detik di background thread untuk fase Trade/Build. Dibuat via TurnTimer.createWithJavaFxDispatch() (Factory Method). Setiap tick memanggil callback onTick dan onTimeout yang di-dispatch ke JavaFX Application Thread via Platform.runLater(), memungkinkan update UI yang aman tanpa memblokir antarmuka.
- GameEngine (src/main/java/engine/GameEngine.java) menyediakan startTradeBuildTimer(), stopTradeBuildTimer(), pauseTimer(), resumeTimer(), dan shutdown() untuk mengelola siklus hidup timer. shutdown() wajib dipanggil saat game selesai untuk menghentikan background thread dan mencegah memory leak.

5.11. Assertion

Lima assertion krusial yang diimplementasikan pada fungsi-fungsi pengubah game state:

- assert currentPhase == TurnPhase.ROLL_DICE pada GameEngine.rollDice() (src/main/java/engine/GameEngine.java): menjaga invariant dadu hanya bisa dilempar pada fase yang benar, mencegah distribusi resource di luar giliran.
- assert !cardPlayedThisTurn pada GameEngine.playCard() (src/main/java/engine/GameEngine.java): menjaga invariant maksimal 1 kartu temuan per giliran sesuai aturan permainan.

- assert players.size() >= 3 && players.size() <= 4 pada constructor GameStateImpl (src/main/java/engine/GameStateImpl.java): memvalidasi jumlah pemain saat inisialisasi, mencegah state tidak valid sejak awal.
- assert amount > 0 pada PlayerInventory.addResource() dan removeResource() (src/main/java/model/player/PlayerInventory.java): menjaga invariant perubahan resource selalu positif, mencegah inventory korup.
- assert deck.size() == 25 pada CardFactory.createStandardDeck() (src/main/java/model/card/CardFactory.java): memvalidasi deck standar selalu 25 kartu, menjaga integritas distribusi kartu.

6. Bonus Yang dikerjakan

Kelompok kami mengerjakan dua fitur bonus, yaitu Generator Peta Baru dan Pemain Bot. Kedua fitur ini memakai sistem plugin berbasis file .jar. Game memuat plugin pada tahap awal permainan, tepatnya di lobby atau setup, sebelum sesi permainan dimulai. Aturan ini sesuai dengan spesifikasi bonus, karena plugin map dan plugin bot tidak boleh dimuat di tengah permainan. Kami membuat sistem plugin dengan URLClassLoader, Reflection, interface, generic class, dan exception khusus. Game membaca file .jar, mencari class yang cocok dengan interface yang dibutuhkan, membuat instance class tersebut, lalu memakai objek hasil loading di dalam game. Dengan mekanisme ini, game dapat menerima peta baru dan strategi bot baru tanpa mengubah alur utama permainan.

6.1. Pemain Bot

Pada implementasi bonus Pemain Bot, interface PlayerStrategy memiliki method takeTurn(GameState state, Player self) dan getStrategyName(). Kami menambahkan parameter Player self agar strategi bot mengetahui pemain yang sedang dikendalikan. Dengan parameter ini, bot dapat membaca kondisi dirinya sendiri, seperti resource, warna, bangunan, dan status permainan. Class BotPlayer merepresentasikan pemain bot. Class ini mewarisi HumanPlayer, sehingga sistem tetap dapat memperlakukan bot sebagai pemain biasa. Perbedaannya, BotPlayer menyimpan objek PlayerStrategy. Objek strategi ini menentukan cara bot mengambil keputusan.

Saat giliran bot dimulai, game memanggil executeTurn(GameState state, GameEngine engine). Di dalam method tersebut, BotPlayer memanggil strategy.takeTurn(state, this). BotPlayer tidak menyimpan logika keputusan secara langsung. Class ini hanya menjadi penghubung antara sistem player dan strategi bot. Untuk memuat strategi bot dari file .jar, kami membuat BotPluginLoader. Class

tersebut mewarisi PluginLoader<PlayerStrategy>. Loader membaca file .jar, mencari class yang mengimplementasikan PlayerStrategy, lalu membuat instance strategi yang valid.

Method loadAllStrategies() memuat semua strategi bot dari file .jar. Jika loader tidak menemukan implementasi PlayerStrategy, sistem melempar PluginLoadException. Jika strategi berhasil dimuat, loader mencatat nama strategi melalui getStrategyName(). Nama strategi ini dapat ditampilkan di UI atau dicatat pada log sistem.

Selanjutnya, bonus ini juga menerapkan beberapa konsep OOP. Interface muncul pada PlayerStrategy, polymorphism muncul saat game memanggil takeTurn() melalui tipe interface, inheritance muncul pada BotPluginLoader yang mewarisi PluginLoader<T> dan BotPlayer yang mewarisi HumanPlayer, lalu composition muncul karena BotPlayer memiliki objek PlayerStrategy sebagai bagian dari dirinya. Generic digunakan pada PluginLoader<PlayerStrategy>, sedangkan PluginLoadException dipakai untuk memisahkan error plugin dari error game biasa. Desain ini membuat pengembang dapat membuat bot baru, misalnya bot yang fokus membangun, bot yang sering melakukan trade ke bank, atau bot yang agresif membeli kartu sehingga game utama tidak perlu diubah selama strategi baru tersebut mengikuti interface PlayerStrategy.

6.2. BGM & SFX Engine (Audio Player)

Implementasi audio dilakukan menggunakan javafx.scene.media.AudioClip dari JavaFX, yang memungkinkan pemutaran audio secara asinkron tanpa memblokir UI thread. Audio diintegrasikan langsung di dua komponen UI yang relevan, tanpa class AudioEngine terpisah.

MainMenuView (com/bananarepublic/view/MainMenuView.java) memuat SFX whoosh.mp3 saat inisialisasi layar melalui method preloadWhoosh(). Audio ini diputar setiap kali karakter Nimon terbang melintasi layar main menu. AudioClip di-preload agar pemutaran langsung tanpa latensi, dengan volume dikonfigurasi ke 0.65 via setVolume().

DeclareVictoryOverlay (com/bananarepublic/view/DeclareVictoryOverlay.java) memuat SFX hooray.mp3 melalui preloadVictoryAudio(). Audio ini dikonfigurasi dengan setCycleCount(AudioClip.INDEFINITE) dan setVolume(0.8), sehingga berputar terus-menerus selama layar kemenangan ditampilkan dan dihentikan via stop() saat overlay ditutup. Kedua pemutaran audio bersifat non-blocking karena AudioClip.play() langsung kembali ke caller tanpa menunggu audio selesai.

6.3. Animasi Background

Animasi background diimplementasikan di dua layar, seluruhnya menggunakan JavaFX Animation API (TranslateTransition, RotateTransition, ScaleTransition, FadeTransition, Timeline) yang berjalan di JavaFX Application Thread sehingga tidak mengganggu logic game.

Pada MainMenuView, terdapat dua jenis animasi yang berjalan terus-menerus. Pertama, empat karakter Nimon (floatMinionView, duckMinionView, snorkelMinionView, waterMinionView) mengambang di area lautan layar main menu. Setiap karakter dianimasikan dengan TranslateTransition untuk gerakan naik-turun (bob) dan RotateTransition untuk goyang kecil (wobble), keduanya dengan setCycleCount(Animation.INDEFINITE) dan setAutoReverse(true). Fase antar karakter dibedakan menggunakan PauseTransition delay agar gerakan tidak serentak. Kedua, tiga karakter terbang (angelMinionFly, birdMinionFly, witchMinionFly) diluncurkan secara random melalui scheduleFlyingMinions(). Sebuah Timeline dijadwalkan ulang secara rekursif dengan interval acak 3.5–8 detik; setiap peluncuran memilih satu karakter yang sedang tidak tampil, lalu menggerakkannya melintasi layar dari kiri ke kanan menggunakan TranslateTransition dengan durasi 3.8–5.8 detik. SFX whoosh diputar bersamaan.

Pada DeclareVictoryOverlay, ketika pemain menang ditampilkan animasi celebrasi. Dua karakter Nimon (minionDanceImageView, minionGuitarImageView) bergerak ke kiri-kanan secara berlawanan menggunakan TranslateTransition dengan INDEFINITE cycle, dikombinasikan ScaleTransition untuk efek memantul. Selain itu, efek kembang api diimplementasikan via spawnFireworksBurst() yang dipanggil oleh Timeline setiap 0.85 detik. Setiap burst menelurkan 14–19 partikel Circle berwarna-warni (warna di-generate dari HSB random), masing-masing dianimasikan dengan kombinasi TranslateTransition (melesat ke arah radial), ScaleTransition (membesar), dan FadeTransition (memudar), semua berdurasi ~1.2–1.8 detik. Node partikel dihapus dari scene graph setelah animasi selesai untuk menjaga memori tetap efisien.

7. Pembagian Tugas

Berikut pembagian tugas masing-masing anggota kelompok FRP:

- **13524098 / Reva Natania Sitohang**

  - Merancang dan mengimplementasikan seluruh package model: model.board, model.building, model.card, model.exception, model.player, model.resource
  - Mendefinisikan semua interface dan enum dasar: Player, Bank, Building, ExperimentCard, MapGenerator, GameStateContext, ResourceType, TerrainType, HarborType, dll.
  - Mengimplementasikan StandardMapGenerator dan IntersectionPathBuilder untuk peta heksagonal standar
  - Membuat UI
- **13524006 / Gabriella Botimada Lubis**
  - Merancang dan mengimplementasikan seluruh package engine: GameEngine, GameStateImpl, phase handlers, ResourceDistributor, BuildValidator, RobberManager, RoadManager (DFS), VictoryManager
  - Mengimplementasikan TurnTimer dengan background thread dan JavaFX Platform.runLater() dispatch
  - Menulis assertion krusial pada fungsi-fungsi pengubah game state
- **13524032 / Juan Oloando Simanungkalit**
  - Merancang dan mengimplementasikan package plugin: PluginLoader, CardPluginLoader, MapPluginLoader, BotPluginLoader, PlayerStrategy, BotPlayer, EventBus, GameEvent
  - Mengimplementasikan package persistence: SaveLoadService, JsonSaveLoadService, SaveData, SaveCorruptedException
  - Mengimplementasikan fitur bonus: Generator Peta (MapPluginLoader) dan Pemain Bot (BotPlayer, BotPluginLoader, PlayerStrategy)
- **13524064 / Stefani Angeline Oroh**
  - Merancang dan mengimplementasikan seluruh layer UI: FXML controllers, HexRenderer, semua dialog controller (Trade, Experiment Cards, Settings & Plugins, Victory, Steal, Discard, dll.)
  - Mengimplementasikan trade logic
  - Menghubungkan UI ke EventBus dan GameController
- **13524093 / Reinsen Silitonga**
  - Mengintegrasikan semua komponen: GameFactory, GameController (bridge engine-UI), App.java entry point, pom.xml configuration
  - Melakukan review merge, quality audit OOP (design pattern enforcement, assertion verification, SOLID compliance)
  - Menyusun laporan tugas besar dan diagram-diagram
  - Membuat UI